# HW3

- 姓名：陈雨萱
- 学号：23300240034
- 专业：计算机科学与技术（拔尖人才计划）

# 关键技术实现

HW3主要是实现一个简单的从AST到IR Tree的翻译器。具体来说：

- 函数fdmjTypeToTreeType做了一个类型转换，将AST层的类型TypeKind转换为IR层类型Type。
- generate_class_table可以根据语义分析阶段生成的Name_Maps，构建一个类级别的符号表Class_table。它会遍历程序中的所有类，为每个类的成员变量分配一个“位置偏移”存入var_pos_map，同时为方法分配一个唯一编号存入method_pos_map。
- generate_method_var_table负责把“变量名”映射成“临时寄存器Temp”。具体来说，它会先遍历当前方法的所有局部变量，为每个变量调用tm->newtemp()分配一个唯一的temp，并记录在var_temp_map中；同时记录变量类型到var_type_map。接着它处理函数参数，为尚未分配temp的参数分配新的temp，但如果参数名和局部变量重名，则跳过它，因为局部变量应覆盖参数。
- ast2tree是将AST转换为IR的“入口函数”。它创建了一个 AST2TreeVisitor，把语义分析结果semant_map和类表class_table挂到visitor 上，然后调用prog->accept(v)遍历AST。遍历结束后，通过v.getTree()取出最终生成的IR树，并强制转换为tree::Program返回。
- visit(fdmj::Program*)处理AST的最外层Program。它先做空指针检查和main方法检查，然后直接调用node->main->accept(*this)，把翻译任务交给MainMethod。所以其实这一层几乎不生成任何IR，它只是一个“转发节点”，最终的IR Program实际上是在MainMethod里构造出来的。
- visit(fdmj::MainMethod*)负责把一个main函数完整翻译成IR。首先确定当前类名和方法名（默认是**$main**^main，必要时从语义信息中回退到已有定义），然后初始化 Temp_map 和方法级变量表，将源程序中的变量映射为 IR 中的 temp；接着遍历 main 方法中的每一条语句，递归调用 visitor 将其翻译为 IR 语句并收集成一个顺序执行的 Seq；在此基础上，构造函数名、统计当前使用的最大 temp 和 label 编号，并将语句序列封装成一个 FuncDecl，再进一步封装为 Program 作为最终 IR 结果返回；最后清理 continue_label 和 break_label，以避免后续资源管理问题。
- visit(fdmj::Nested*)用于处理一组嵌套语句，将AST中的语句列表逐条翻译成IR语句：首先做空检查，然后遍历node->sl中的每一条语句，对每条语句递归调用accept生成对应的IR tree::Stm，并收集到一个vector中，最后将这些语句封装成一个tree::Seq，表示顺序执行的一段IR；如果语句块为空，则返回空结果。
- visit(fdmj::If*)实现了if-else控制流到IR的翻译。首先将条件表达式翻译为Tr_cx形式，来表示带有真假出口的条件跳转，然后先递归翻译then和else分支，以保证label分配顺序一致，之后再统一申请then_label、else_label和join_label，并用patch技术把条件的true/false分支分别指向then和else；

最后按顺序拼接IR：先执行条件跳转，再进入then分支，若存在else则在then结束后跳转到join，随后执行else分支，最后汇合到join_label；如果没有else，则直接把false分支作为结束点。

- visit(fdmj::While*)负责将while循环翻译为IR。首先保存原有的continue和break标签用于嵌套循环恢复，然后先翻译循环条件为Tr_cx以保证label编号顺序正确，接着再申请start_label作为循环入口、body_label作为循环体和exit_label作为退出点，并将条件的true分支patch到body、false分支patch到exit；随后翻译循环体语句，并按顺序构造IR：入口label → 条件判断 → body label → 循环体 → 跳回入口 → exit label；同时更新continue_label指向循环入口、break_label 指向退出点，使得循环内部的continue/break能正确跳转；最后恢复旧的标签环境。
- Assign/Continue/Break/Return的Visitor都是对都是控制流或基本语句从AST到IR的直接翻译。Assign分别翻译左值和右值表达式，通过accept得到Tr_Exp，再统一转成Tr_ex即“可计算表达式”，最后生成一条tree::Move(dst, src)，表示赋值语句。而Continue和Break则依赖在while中维护的continue_label和break_label，直接翻译为Jump，体现了结构化控制流转换到goto的过程。Return 则是将返回表达式翻译为IR表达式，构造tree::Return，如果没有返回值则默认返回0，保证IR语义完整。
- PutInt/PutCh/Starttime/Stoptime/CallStm其实都是函数调用/外部调用从AST到IR的翻译。他们都先把参数表达式翻译成IR表达式，然后封装成tree::ExtCall，最后再包一层tree::ExpStm表示“有副作用的表达式语句”。其中PutInt和PutCh是特化的系统调用，而CallStm是通用版本，可以处理任意函数名+参数列表。
- visit(fdmj::BinaryOp*)实现了二元表达式从AST到IR的统一翻译逻辑，根据操作符类型将表达式转换为不同形式的中间表示：对于算术运算，它递归翻译左右子表达式并通过unEx转为可求值表达式，直接构造tree::Binop，最终返回一个Tr_ex；对于逻辑运算，则采用短路求值+回填技术，先将左右表达式转换为条件形式Tr_cx，通过创建中间标签并对true_list或false_list进行patch，实现“按需计算右侧表达式”的控制流拼接，同时合并对应的真假链表，最终返回新的Tr_cx；对于关系运算，则生成一个tree::Cjump，并构造对应的真假跳转链表，表示该表达式本质上是一个控制流分支而非立即求值的数值表达式。
- visit(fdmj::Unary*)实现了一元运算的IR翻译。对于取负运算-e，它先将内部表达式转换为Tr_ex，然后构造一个0-e的tree::Binop，避免单独设计一元运算节点；而对于逻辑非!e，则将表达式转为条件形式Tr_cx，通过交换true_list和false_list来实现布尔取反，而不直接生成数值。
- visit(fdmj::IdExp*)将变量引用翻译为IR中的临时变量访问，其本质是一个符号表查询+Temp映射过程。它从method_var_table中根据变量名查找对应的Temp，然后构造一个tree::TempExp表示该变量的值，并封装为Tr_ex返回。如果变量不存在则直接报错退出，避免生成非法IR。
- visit(fdmj::IntExp*)用于处理整数字面量。它直接将AST中的整数值封装为tree::Const，并包装成Tr_ex返回，表示这是一个可以直接求值的表达式。
- visit(fdmj::CallExp*)实现了对函数调用表达式的IR翻译。它首先进行基本合法性检查，函数名是否存在、是否是对象调用，然后提取函数名作为外部调用标识；接着对参数列表逐个递归翻译，并统一通过unEx转换为可求值表达式，按顺序加入参数数组中；最后构造一个tree::ExtCall表示函数调用，并封装为Tr_ex返回。
- GetInt/GetCh的Visitor直接构造对"getint"或"getch"的ExtCall，并封装为Tr_ex返回，表示这是一个有返回值的表达式。

# 相关Q回答

### Q1.1：Tiger IR+ 中各个类的作用说明

> tree::Program 表示整个程序的IR结构，它的核心成员是一个函数声明列表，也就是将源程序整体转换为多个函数的集合，是IR的最顶层结构。
> tree::FuncDecl 表示一个函数或方法的IR表示，包含函数名（已经重命名为类名+方法名）、参数列表（temps形式）、函数体（Stm）、返回类型以及该函数中使用的最大temp和label编号，用于后续代码生成阶段。
> tree::Stm 是所有“语句节点”的抽象基类，表示IR中的控制流或副作用操作（如跳转、赋值等），所有具体语句如Jump、Move等都继承自它。
> tree::Seq 表示一组顺序执行的语句序列，是IR中最常见的结构，用于把多个语句按顺序组织起来，对应源程序中的代码块。
> tree::LabelStm 表示一个标签语句，用于标记程序执行的某个位置，是跳转语句Jump/Cjump的目标，类似汇编中的label。
> tree::Jump 表示无条件跳转语句，执行后直接跳转到指定的label，用于实现循环回跳、break/continue等控制流。
> tree::Cjump 表示条件跳转语句，根据左右表达式的比较结果，跳转到true标签或false标签，是if、while等控制结构的核心实现方式
> tree::Move 表示赋值操作，将源表达式的值赋给目标表达式，是IR中变量赋值和内存写入的统一表示。
> tree::ExpStm 用于将一个表达式作为语句执行并忽略其返回值，常用于函数调用等只关心副作用的表达式。
> tree::Return 表示函数返回语句，包含一个返回值表达式，对应源语言中的return语句。
> tree::Exp 是所有表达式节点的抽象基类，表示可以计算出值的IR节点，每个表达式都有类型。
> tree::Binop 表示二元运算表达式，包含操作符以及左右操作数，是算术和逻辑计算的核心结构。
> tree::Mem 表示内存访问操作，即访问某个地址对应的内存内容，常用于数组访问或指针解引用。
> tree::TempExp 用于将一个临时变量包装成表达式，是变量在IR中的基本表示形式，可以理解为“寄存器访问”。
> tree::Eseq 表示“先执行语句，再计算表达式”的结构，用于在表达式中嵌入副作用，是Tiger IR中处理复杂表达式的重要机制。
> tree::Name 用于将label或字符串标签转换为地址，通常用于函数调用或跳转目标的表示。
> tree::Const 表示整型常量，是最基本的表达式形式，用于表示数字字面量。
> tree::Call 表示函数调用或方法调用，包含函数名、对象以及参数列表，在支持类的情况下用于方法调用。
> tree::ExtCall 表示对外部函数的调用，区别于普通Call，它不依赖对象，是与运行时库交互的接口。

### Q1.2：Tiger IR+比Tiger IR多了什么

Tiger IR+引入了Program和FuncDecl这样的顶层结构，用来显式表示“程序由一组函数组成”。在虎书中的Tiger IR主要是围绕表达式Exp展开，而没有一个明确的“函数列表”结构；而在FDMJ中，程序是由多

个method构成的，因此需要通过Program和FuncDecl来统一表示整个程序。IR+更适合后续编译也更贴近实际语言的函数式结构。并且IR+ 增强了函数表示能力，例如在FuncDecl中增加了参数列表、返回类型、以及 last_temp_num 和 last_label_num 等信息。这些内容在虎书中没有，但在实际编译过程中是必须的：参数列表用于表示函数调用时的输入，返回类型用于类型检查和后续生成目标代码，而temp和label的编号范围则有助于寄存器分配和标签管理。所以IR+能够承载更完整的函数语义信息。在语句层面，IR+增加了一些更贴近高级语言语义的结构，例如Return语句。在虎书中，函数返回通常通过表达式间接表示，而在FDMJ中，return是一个明确的语句，因此需要在IR中显式建模。这使得AST到IR的翻译更加直接，也减少了额外的转换复杂度。IR+还引入了ExtCall这一结构，用于表示对外部函数的调用。在虎书中函数调用统一用Call表示，对“用户定义函数”和“外部函数”进行区分，有助于后续代码生成阶段对不同调用类型进行不同处理。IR+在类型系统上做了简化和明确化，引入Type枚举，并让每个表达式节点都携带类型信息。在FDMJ中，由于涉及指针（如数组、对象）等概念，必须在IR层区分整数和指针类型。因此，这一扩展是为了支持更复杂的数据结构和内存操作。整体来看，通过增加 Seq、Return、Jump、Cjump 等语句结构，并允许在表达式中嵌入语句，Tiger IR+实现了对控制流的精确表达，使得if、while、break、continue 等结构都可以自然地翻译成IR。这种扩展是为了弥合Tiger IR与实际命令式语言之间的语义差距。

**Q2：不带class，如何完成各类语法成分的翻译**

在不带class的AST到IR翻译中，我基于IR+结构将不同语法成分分别映射为对应的IR结点，并通过Tr_Exp统一管理表达式在不同上下文下的翻译方式。对于**算术运算**，我将其翻译为tree::Binop结点。具体做法是先递归翻译左右子表达式，并通过unEx获取其值表达式tree::Exp，然后构造一个Binop节点表示该运算，最终封装为Tr_ex返回。这种方式直接对应IR中的二元运算，结构清晰。对于**比较运算**，我将其翻译为条件跳转形式，即tree::Cjump。具体来说，会先翻译左右表达式得到两个tree::Exp，然后构造Cjump节点，并维护对应的true_list和false_list。这种翻译方式不会立即产生布尔值，而是生成控制流分支，便于后续在if/while中进行回填。对于**逻辑运算**，我实现了短路求值。具体做法是先将左右表达式转换为Tr_cx，然后通过插入中间label，将左表达式的真假分支与右表达式的执行连接起来。例如对于&&，只有当左边为真时才执行右边；对于||，只有当左边为假时才执行右边。最终通过拼接tree::Seq 构造完整控制流，并维护新的true_list和false_list。对于**赋值语句**，我将其翻译为tree::Move节点。具体流程是：先翻译左值表达式，通过unEx得到目标地址或temp；再翻译右值表达式得到源值；最后构造Move(dst, src)表示赋值操作。这种方式统一了变量赋值的IR表示。对于**条件语句**，我采用基于Cjump和label的控制流构造方式。首先翻译条件表达式为Tr_cx，然后分别翻译then分支和else分支，接着创建then_label、else_label和join_label。通过回填条件的true/false跳转目标，并用LabelStm和Jump将各部分连接起来，最终形成一段Seq表示完整的if控制流结构。对于**循环语句**，我同样使用label和跳转构造控制流。具体来说，会创建start_label、body_label和 exit_label，并将条件表达式翻译为Tr_cx。条件为真跳转到循环体，为假跳转到退出位置；循环体执行后再跳回入口。同时，通过维护continue_label和break_label，支持循环内部的continue和break控制。对于break和continue语句，我分别将其翻译为跳转到当前循环的break_label和continue_label，即生成tree::Jump节点。这依赖于在翻译while时维护的上下文标签，从而实现正确的控制流跳转。对于return语句，我将其翻译为tree::Return节点。如果return后有表达式，则先翻译该表达式并作为返回值；如果没有，则默认返回常数0。这种设计使函数返回在IR中有明确表示。对于**函数调用**，我统一使用 tree::ExtCall表示。具体做法是：先翻译所有参数表达式并收集为参数列表，然后构造ExtCall(extfun, args)。如果调用出现在语句位置，则再用ExpStm包裹以忽略返回值；如果在表达式中，则直接作为Tr_ex返回。这

种方式适用于 getint、putint等运行时函数。对于**变量访问**，我通过method_var_table查找变量对应的Temp，并构造tree::TempExp表示变量值。这实现了从变量名到IR临时变量（寄存器抽象）的映射。对于**常量**，直接翻译为tree::Const节点，表示整数常量。注意：在HW3中不涉及数组和类，因此数组初始化、访问以及类相关操作未在本次实现中处理。

# 开发过程 & 测试结果

在HW3的实现过程中，整体功能逻辑比较简单，在初步完成AST到IR的转换后，在debug过程重点处理了三个地方：其一是，按照文档和老师所给示例的要求，while的所有的语句应该在同一个sequence里面，但我在最初实现while的Visitor时，构造IR时多包了一层Sequence，写成了这样：

```
Seq(
  Label,
  Seq(
    CJump,
    ...
  )
)
```

于是改掉了，去掉了那层sequence。

其二是我最初写的whlie的Visitor在分配label的时候顺序上与老师有出入。一开始我的写法是：

```
Label* start = newlabel();
Label* body  = newlabel();
Label* exit  = newlabel();

cond = translate(cond);  // 在这里内部才会产生label
```

这种方法的编号就是:

```
start=100, body=101, exit=102
cond内部=103,104,...
```

按理来说只要编译正确，这是无伤大雅的，但是我觉得下面这种分配方式更合理一些：

> 先翻译cond，内部消耗label
> 再 new start/body/exit

所以我改成了：

```
cond = translate(cond);  // 先消耗 label

Label* start = newlabel();
Label* body  = newlabel();
Label* exit  = newlabel();
```

其三是我的if的Visitor，对label的分配和老师也有出入，同理进行了调整。我一开始同样是这样处理：

```
Label* then = newlabel();
Label* else = newlabel();
Label* join = newlabel();

cond = translate(cond);
```

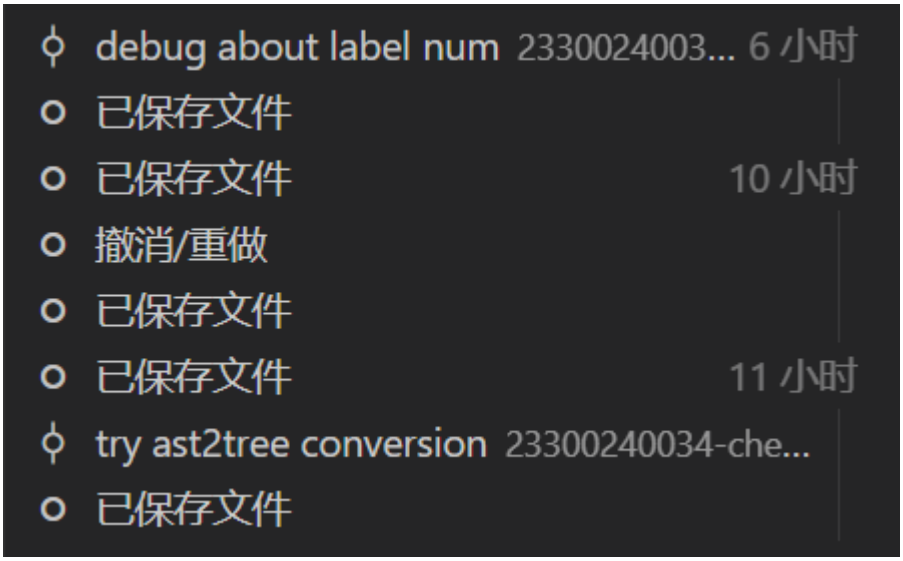
但是其实老师示例的思路是：

```
cond = translate(cond);  // 先翻译条件

Label* then = newlabel();
Label* else = newlabel();
Label* join = newlabel();
```

于是改掉了。（我其实是写了git的commit信息记录开发过程的，但是我忘记gitee不能push上去，于是遗憾离场，但是timeline能看到我commit两次的结果）

| Graph | Description | Date | Author | Commit |
|---|---|---|---|---|
| | master HW3 finished | 23 Mar 2026 20:02 | 23300240034-chen... | 084fbca8 |
| | origin/HEAD origin/master HW3 tests added | 17 Mar 2026 17:55 | 王晓阳 | 67c62231 |
| | HW3 files | 15 Mar 2026 23:43 | 王晓阳 | 1c436d1c |
| | HW3 files | 15 Mar 2026 23:40 | 王晓阳 | 27f32a0e |




测试结果：
得到的测试结果与老师的示例完全一致。

```
Reading irtest1
------Reading AST from : irtest1.2-semant.ast------------
Saving AST (XML) to: irtest1.2-semant-debug.ast
Converting AST to IR
Compiler Configuration:: address_length: 4; memory_alignment: 4; int_length: 4; float_length: 4; double_length: 8
Saving IR (XML) to: irtest1.3.irp
-----Done---
Reading irtest1
------Reading AST from : irtest1.2-semant.ast------------
Saving AST (XML) to: irtest1.2-semant-debug.ast
Converting AST to IR
Compiler Configuration:: address_length: 4; memory_alignment: 4; int_length: 4; float_length: 4; double_length: 8
Saving IR (XML) to: irtest1.3.irp
-----Done---
Reading irtest2
------Reading AST from : irtest2.2-semant.ast------------
Saving AST (XML) to: irtest2.2-semant-debug.ast
Converting AST to IR
Compiler Configuration:: address_length: 4; memory_alignment: 4; int_length: 4; float_length: 4; double_length: 8
Saving IR (XML) to: irtest2.3.irp
-----Done---
Reading irtest2
------Reading AST from : irtest2.2-semant.ast------------
Saving AST (XML) to: irtest2.2-semant-debug.ast
Converting AST to IR
Compiler Configuration:: address_length: 4; memory_alignment: 4; int_length: 4; float_length: 4; double_length: 8
Saving IR (XML) to: irtest2.3.irp
-----Done---
Reading irtest3
------Reading AST from : irtest3.2-semant.ast------------
Saving AST (XML) to: irtest3.2-semant-debug.ast
Converting AST to IR
Compiler Configuration:: address_length: 4; memory_alignment: 4; int_length: 4; float_length: 4; double_length: 8
Saving IR (XML) to: irtest3.3.irp
-----Done---
Reading irtest3
------Reading AST from : irtest3.2-semant.ast------------
Saving AST (XML) to: irtest3.2-semant-debug.ast
Converting AST to IR
Compiler Configuration:: address_length: 4; memory_alignment: 4; int_length: 4; float_length: 4; double_length: 8
Saving IR (XML) to: irtest3.3.irp
-----Done---
Reading irtest4
------Reading AST from : irtest4.2-semant.ast------------
Saving AST (XML) to: irtest4.2-semant-debug.ast
Converting AST to IR
Compiler Configuration:: address_length: 4; memory_alignment: 4; int_length: 4; float_length: 4; double_length: 8
```

```
------Reading AST from : irtest5.2-semant.ast------------
Saving AST (XML) to: irtest5.2-semant-debug.ast
Converting AST to IR
Compiler Configuration:: address_length: 4; memory_alignment: 4; int_length: 4; float_length: 4; double_length: 8
Saving IR (XML) to: irtest5.3.irp
-----Done---
Reading irtest6
------Reading AST from : irtest6.2-semant.ast------------
Saving AST (XML) to: irtest6.2-semant-debug.ast
Converting AST to IR
Compiler Configuration:: address_length: 4; memory_alignment: 4; int_length: 4; float_length: 4; double_length: 8
Saving IR (XML) to: irtest6.3.irp
-----Done---
Reading irtest6
------Reading AST from : irtest6.2-semant.ast------------
Saving AST (XML) to: irtest6.2-semant-debug.ast
Converting AST to IR
Compiler Configuration:: address_length: 4; memory_alignment: 4; int_length: 4; float_length: 4; double_length: 8
Saving IR (XML) to: irtest6.3.irp
-----Done---
Reading irtest7
------Reading AST from : irtest7.2-semant.ast------------
Saving AST (XML) to: irtest7.2-semant-debug.ast
Converting AST to IR
Compiler Configuration:: address_length: 4; memory_alignment: 4; int_length: 4; float_length: 4; double_length: 8
Saving IR (XML) to: irtest7.3.irp
-----Done---
Reading irtest7
------Reading AST from : irtest7.2-semant.ast------------
Saving AST (XML) to: irtest7.2-semant-debug.ast
Converting AST to IR
Compiler Configuration:: address_length: 4; memory_alignment: 4; int_length: 4; float_length: 4; double_length: 8
Saving IR (XML) to: irtest7.3.irp
-----Done---
Reading irtest8
------Reading AST from : irtest8.2-semant.ast------------
Saving AST (XML) to: irtest8.2-semant-debug.ast
Converting AST to IR
Compiler Configuration:: address_length: 4; memory_alignment: 4; int_length: 4; float_length: 4; double_length: 8
Saving IR (XML) to: irtest8.3.irp
-----Done---
Reading irtest8
------Reading AST from : irtest8.2-semant.ast------------
Saving AST (XML) to: irtest8.2-semant-debug.ast
Converting AST to IR
Compiler Configuration:: address_length: 4; memory_alignment: 4; int_length: 4; float_length: 4; double_length: 8
Saving IR (XML) to: irtest8.3.irp
-----Done---
root@cyx:~/fducompilerh2026/fducompilerh2026/HW3#
```

# 参考资料

- 虎书中文版，IR构造相关部分
- 课堂PPT和week3content.md课件
- Chat GPT，主要帮助我处理对IR的理解，以及label分配的问题